

Building Simple UIs for Your Agents with PydanticAI and Streamlit



Launch Intelligence
10.9K subscribers

Join

Subscribe

10



Share

Ask

Download



441 views Sep 4, 2025 Members first on March 20, 2025 #AI #APIs #Python

Join AI Dev Skool & Launch Your AI Startup Today! <https://www.skool.com/ai-software-dev...> is the community for founders, builders, and AI innovators ready to take their projects to the next level.

AI agents are powerful, but without an intuitive UI, they remain stuck in the backend. In this tutorial, we'll explore how to build interactive UIs for PydanticAI agents using Streamlit—one of the easiest ways to create web interfaces for AI applications.

What You'll Learn:

- ✓ Why UI matters for AI agents
- ✓ How to connect PydanticAI with Streamlit
- ✓ Step-by-step coding tutorial for building an agent UI
- ✓ Best practices for making UIs responsive and user-friendly

Why Streamlit?

- Rapid Development – Build UIs with minimal code
- Seamless Integration – Connect Streamlit with PydanticAI models
- Real-time Interactions – Enable instant feedback and debugging

What We're Building:

We'll start with a basic PydanticAI agent, test it in the terminal, and then wrap it with a Streamlit UI for interactive use. By the end, you'll have a fully functional AI-powered app!

Next Video: Building a RAG system with PydanticAI and a vector database.

Full-Stack Agentic AI Masterclass Series:

- Part 1: Introduction
- Part 2: The AI framework landscape in 2025
- Part 3: Building effective agents - principles & practice
- Part 4: Prompt Engineering for effective agents
- Part 5: [Your AI Agents Are Useless Without This: P...](#)
- Part 6: [Building Simple UIs for Your Agents with P...](#)
- Part 7: [Stop Building Dumb Agents: Easy Database C...](#)
- Part 7.5: [AI That Writes SQL: Automate Queries with ...](#)
- Part 8: RAG
- Part 9: Integrating PydanticAI with N8N
- Part 10: Containers and deployment
- Part 11: E2E Final Project - customer support agent with PydanticAI

Full-Stack Agentic AI with PydanticAI

- The framework landscape in 2025
- Building effective agents – principles & practice
- Prompt Engineering for Better Results
- Building agent APIs – PydanticAI, FastAPI
- Building agent UI – PydanticAI, Streamlit
- Building DB agents with PydanticAI
- RAG with PydanticAI
- Integrating PydanticAI with N8N
- Containers and deployment
- Project: Customer support agent w/ PydanticAI



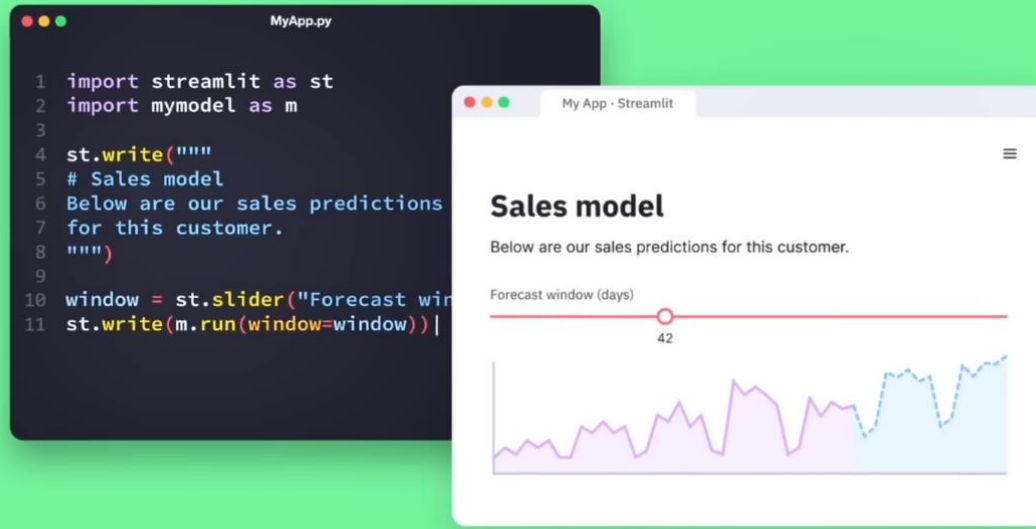
Building Agent UI

Full-Stack Agentic AI Masterclass

AI SOFTWARE DEV

Skool
Channel
Newsletter

What is Streamlit?

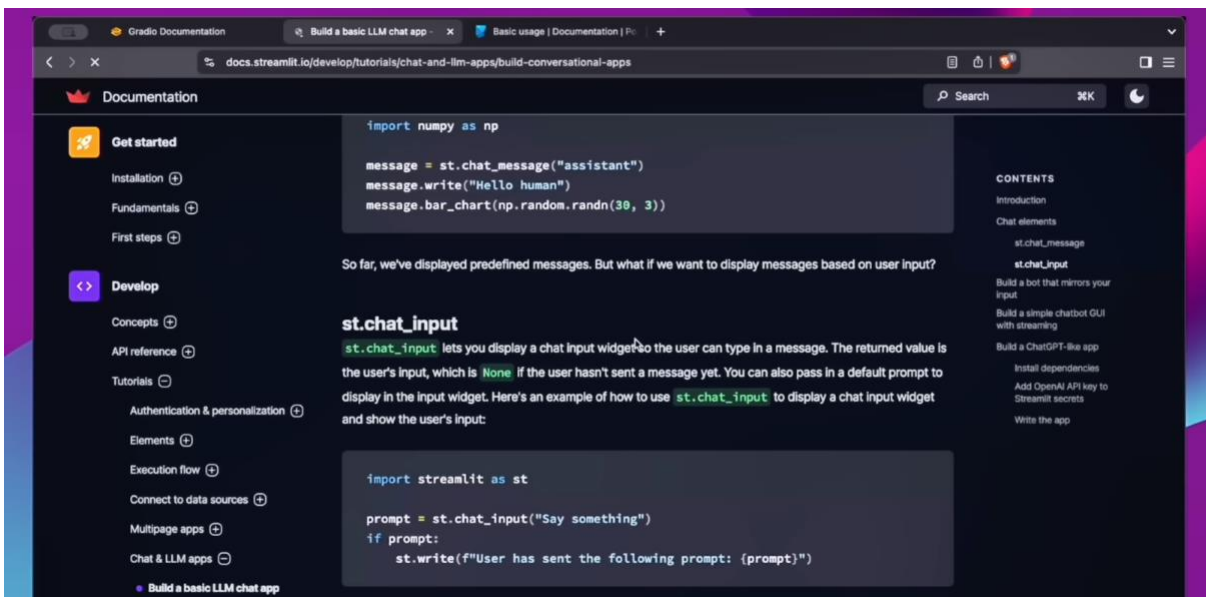
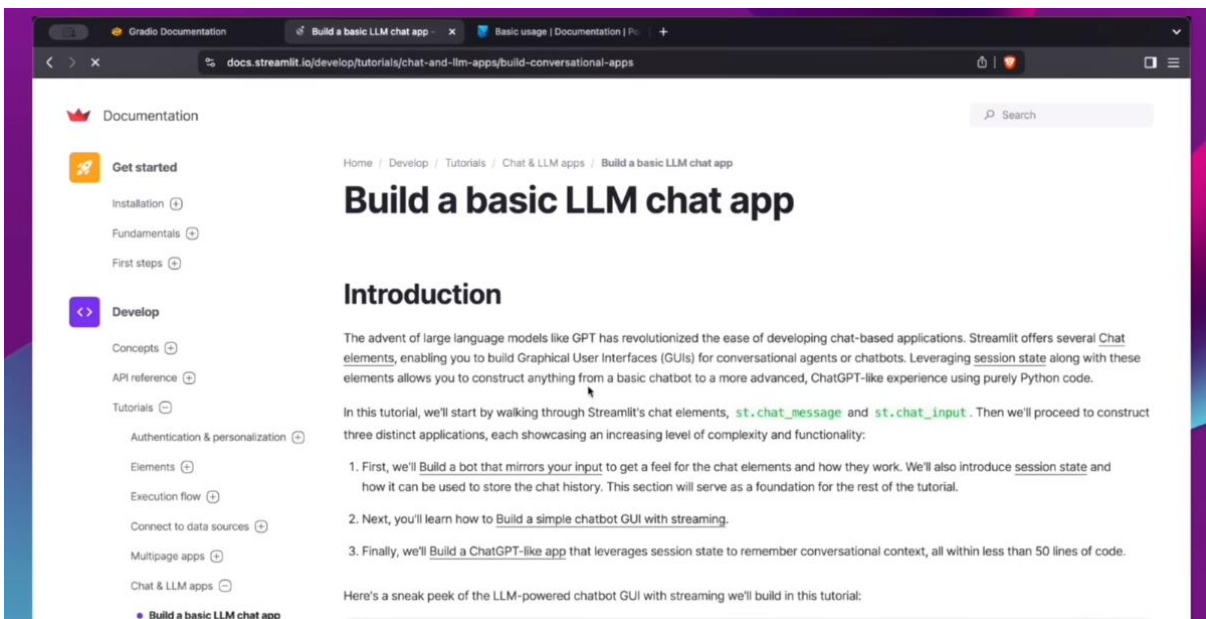
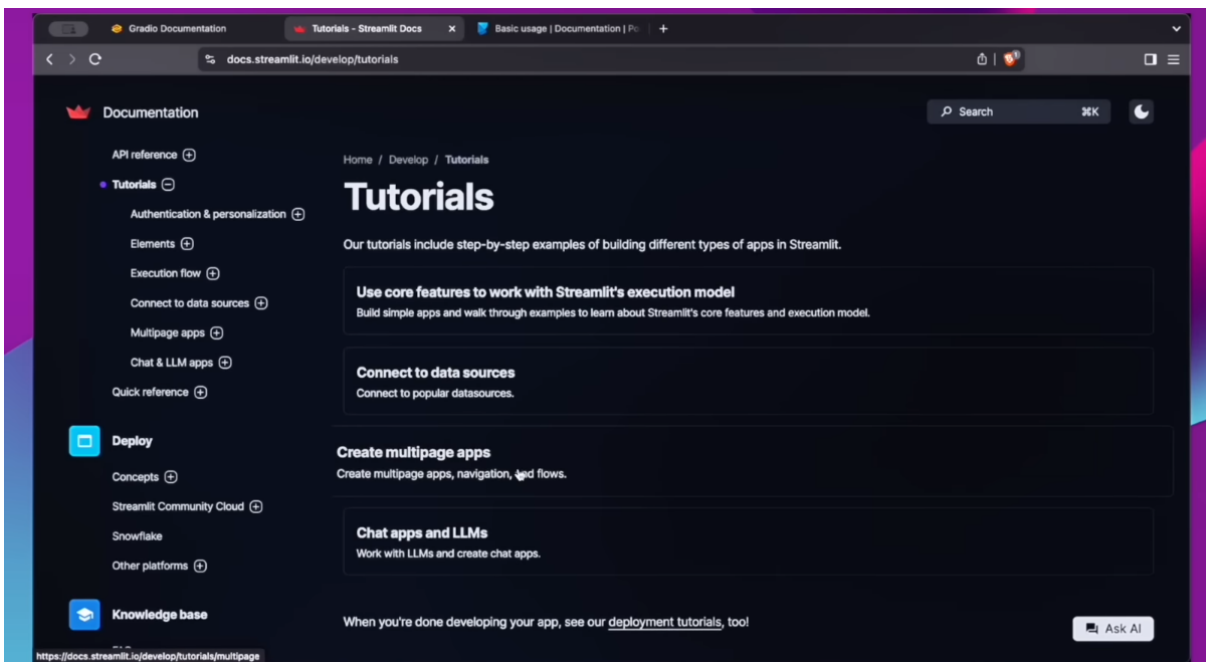


Streamlit Documentation

The screenshot shows the Streamlit documentation website at docs.streamlit.io. The page has a dark theme and a sidebar with navigation links: 'Get started' (Installation, Fundamentals, First steps), 'Develop' (Concepts, API reference, Tutorials, Quick reference), and 'Deploy' (Concepts, Streamlit Community Cloud, Snowflake). The main content area is titled 'Streamlit documentation' and includes a brief description of Streamlit as an open-source Python framework. Below this are three colored boxes: 'Setup and installation' (orange), 'API reference' (purple), and 'App gallery' (blue). At the bottom, there is a section titled 'How to use our docs' with a link to 'Get started with Streamlit!'.

The screenshot shows the 'What's new' page on the Streamlit documentation website. The page features a sidebar with navigation links: 'Get started' (Installation, Fundamentals, First steps), 'Develop' (Concepts, API reference, Tutorials, Quick reference), and 'Deploy'. The main content area is titled 'What's new' and contains six articles, each with an icon and a title:

- File uploader for `st.chat_input`**: Users can upload files directly within `st.chat_input`!
- JSON column configuration**: `st.dataframe` and `st.data_editor` support JSON compatible objects.
- Download buttons without reruns**: You can disable app reruns for `st.download_button`.
- Access the user's timezone**: `st.context` lets you read the user's timezone and timezone offset.
- Dataframe row height**: You can set the height of rows in `st.dataframe` and `st.data_editor`.
- Markdown image captions**: Markdown has a new `:small[]` directive for small text, the same size as `st.caption`.



Why Streamlit?

Benefits and Core Features

1. Benefits

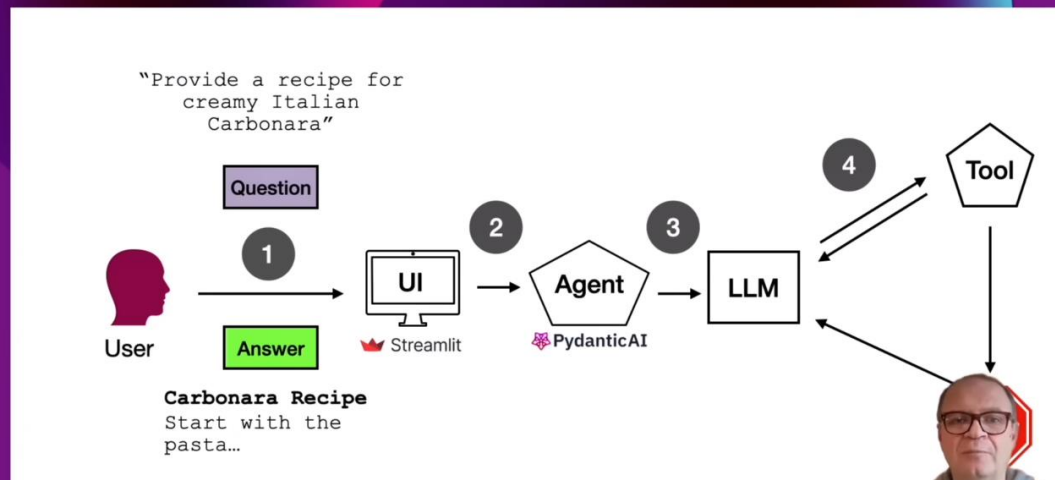
1. **Rapid Development** – Build functional UIs with minimal code.
2. **Seamless Integration** – Easily connect Streamlit with PydanticAI models.
3. **Interactive Debugging** – Quickly test and refine agent outputs.

2. Core Features

1. **Dynamic Forms** – Generate UI inputs from Pydantic models.
2. **Real-time Updates** – Instantly see agent responses.
3. **Customization** – Extend layouts with Streamlit components.



What We're Building



Building Agent UI with Streamlit and PydanticAI

1. Write a **PydanticAI Agent**
2. Test the **PydanticAI Agent** in the terminal
3. Modify the PydanticAI Agent so it can be **imported from Streamlit**
4. Write a **Streamlit client**
5. Launch the **Streamlit server**
6. **Test agent** – perform basic operations



PydanticAI Agent

```
1 import os
2 from colorama import Fore
3 from dotenv import load_dotenv
4 from pydantic_ai.models.openai import OpenAIModel
5 from pydantic_ai import Agent
6
7 load_dotenv()
8
9 # Define the model
10 model = OpenAIModel('gpt-4o-mini', api_key=os.getenv('OPENAI_API_KEY'))
11 agent = Agent(model=model, system_prompt="You are a helpful assistant")
12
13 def get_response(user_input: str) -> str:
14     return agent.run_sync(user_input)
15
16 def get_response_with_hi
```

```
12
13 def get_response(user_input: str) -> str:
14     return agent.run_sync(user_input)
15
16 def get_response_with_history(user_input: str, history: list) -> str:
17     response = agent.run_sync(user_input)
18     return response.data
19
20 # Define the main loop
21 def main_loop():
22     while True:
23         user_input = input(">> I am your assitant. How can I help you today? ")
24         if user_input.lower() in ["quit", "exit", "q"]:
25             print("Goodbye!")
26             break
27
28         # Run the agent
29         result = get_response(user_input)
30         print(Fore.WHITE, result.data)
31
32 # Run the main loop
33 if __name__ == "__main__":
34     main_loop()
```

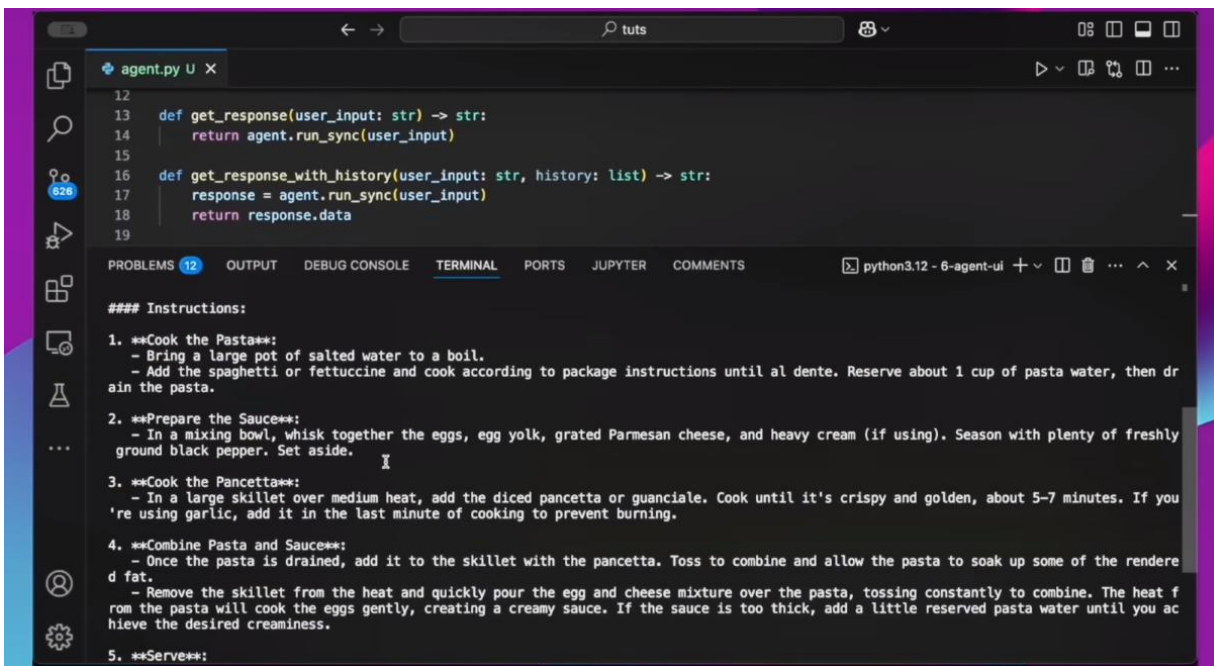
```
PROBLEMS (12) OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS
python3.12 - 6-agent-ui + - - - - -
o (myenv) 18:08:50 ~/dev/tuts/full-stack-ai-masterclass/6-agent-ui $ poetry run python agent.py
>> I am your assitant. How can I help you today? █
```

```
12
13 def get_response(user_input: str) -> str:
14     return agent.run_sync(user_input)
15
16 def get_response_with_history(user_input: str, history: list) -> str:
17     response = agent.run_sync(user_input)
18     return response.data
19
```

```
PROBLEMS (12) OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS
python3.12 - 6-agent-ui + - - - - -
o (myenv) 18:08:50 ~/dev/tuts/full-stack-ai-masterclass/6-agent-ui $ poetry run python agent.py
>> I am your assitant. How can I help you today? Get me a creamy carbonara recipe
Sure! Here's a classic creamy carbonara recipe that serves about 4 people. It's simple and delicious, using traditional ingredients for an authentic flavor.
I
### Creamy Carbonara Recipe

#### Ingredients:
- 400g (14 oz) spaghetti or fettuccine
- 150g (5 oz) pancetta or guanciale, diced
- 2 large eggs
- 1 large egg yolk
- 100g (3.5 oz) grated Parmesan cheese (plus extra for serving)
- 100ml (3.5 fl oz) heavy cream (optional for extra creaminess)
- 2 cloves garlic, minced (optional)
- Freshly ground black pepper
- Salt (for pasta water)
- Fresh parsley, chopped (for garnish, optional)

#### Instructions:
```

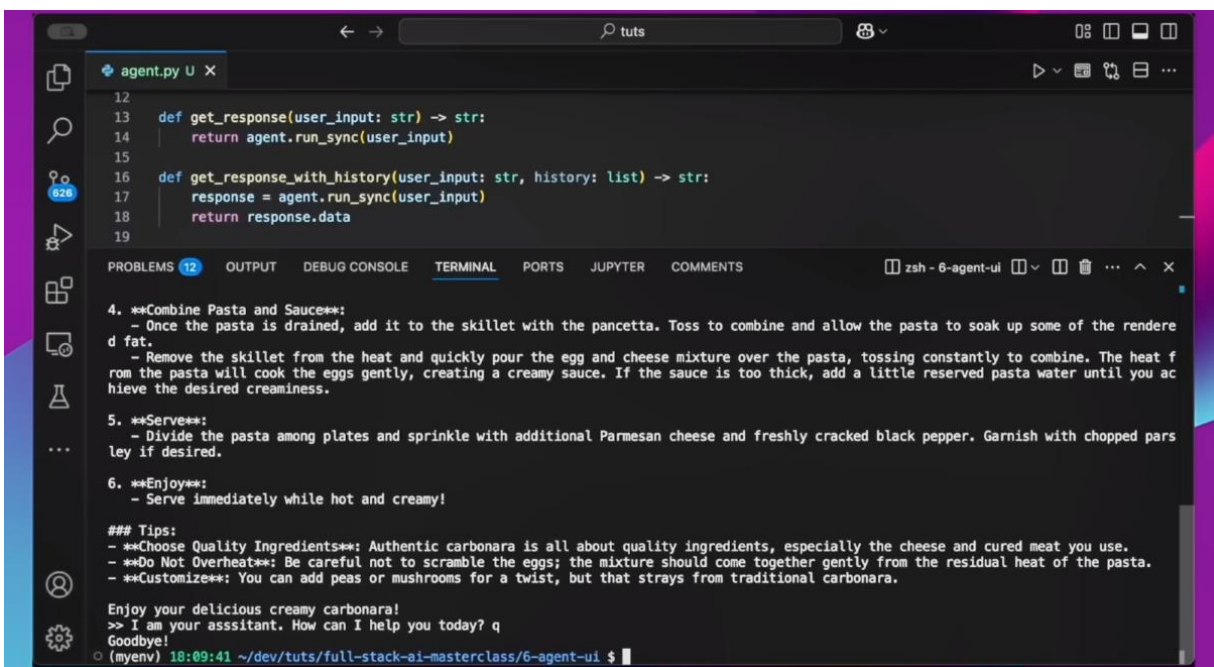


```
12
13 def get_response(user_input: str) -> str:
14     return agent.run_sync(user_input)
15
16 def get_response_with_history(user_input: str, history: list) -> str:
17     response = agent.run_sync(user_input)
18     return response.data
19
```

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS python3.12 - 6-agent-ui

Instructions:

1. ****Cook the Pasta****:
 - Bring a large pot of salted water to a boil.
 - Add the spaghetti or fettuccine and cook according to package instructions until al dente. Reserve about 1 cup of pasta water, then drain the pasta.
2. ****Prepare the Sauce****:
 - In a mixing bowl, whisk together the eggs, egg yolk, grated Parmesan cheese, and heavy cream (if using). Season with plenty of freshly ground black pepper. Set aside.
3. ****Cook the Pancetta****:
 - In a large skillet over medium heat, add the diced pancetta or guanciale. Cook until it's crispy and golden, about 5-7 minutes. If you're using garlic, add it in the last minute of cooking to prevent burning.
4. ****Combine Pasta and Sauce****:
 - Once the pasta is drained, add it to the skillet with the pancetta. Toss to combine and allow the pasta to soak up some of the rendered fat.
 - Remove the skillet from the heat and quickly pour the egg and cheese mixture over the pasta, tossing constantly to combine. The heat from the pasta will cook the eggs gently, creating a creamy sauce. If the sauce is too thick, add a little reserved pasta water until you achieve the desired creaminess.
5. ****Serve****:



```
12
13 def get_response(user_input: str) -> str:
14     return agent.run_sync(user_input)
15
16 def get_response_with_history(user_input: str, history: list) -> str:
17     response = agent.run_sync(user_input)
18     return response.data
19
```

PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS zsh - 6-agent-ui

4. ****Combine Pasta and Sauce****:
 - Once the pasta is drained, add it to the skillet with the pancetta. Toss to combine and allow the pasta to soak up some of the rendered fat.
 - Remove the skillet from the heat and quickly pour the egg and cheese mixture over the pasta, tossing constantly to combine. The heat from the pasta will cook the eggs gently, creating a creamy sauce. If the sauce is too thick, add a little reserved pasta water until you achieve the desired creaminess.
5. ****Serve****:
 - Divide the pasta among plates and sprinkle with additional Parmesan cheese and freshly cracked black pepper. Garnish with chopped parsley if desired.
6. ****Enjoy****:
 - Serve immediately while hot and creamy!

Tips:

- ****Choose Quality Ingredients****: Authentic carbonara is all about quality ingredients, especially the cheese and cured meat you use.
- ****Do Not Overheat****: Be careful not to scramble the eggs; the mixture should come together gently from the residual heat of the pasta.
- ****Customize****: You can add peas or mushrooms for a twist, but that strays from traditional carbonara.

Enjoy your delicious creamy carbonara!

>> I am your assistant. How can I help you today? q

Goodbye!

(myenv) 18:09:41 ~/dev/tuts/full-stack-ai-masterclass/6-agent-ui \$

Next, let us build the UI

Streamlit UI (Streaming)

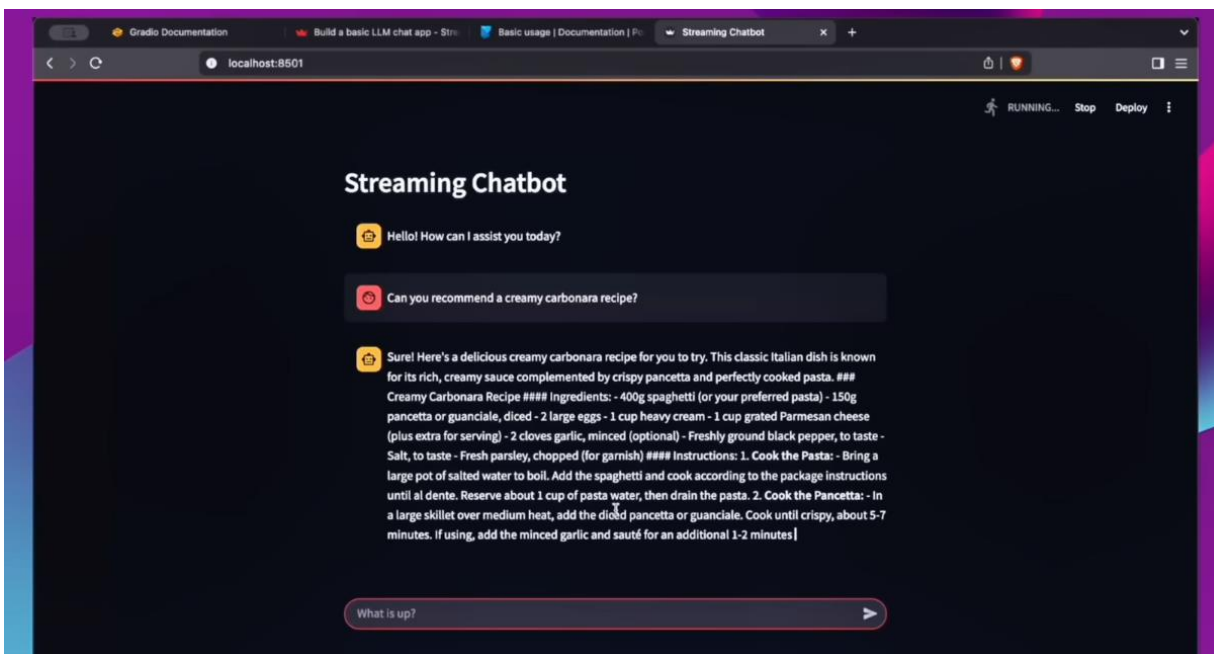
```
1
2 import time
3 import streamlit as st
4 from agent import get_response
5
6 # Constants
7 height = 600
8 title = "Streaming Chatbot"
9 icon = ":robot:"
10
11 # Set page title and icon
12 st.set_page_config(page_title=title, page_icon=icon)
13 st.header(title)
14
15 def generate_message(user_input):
16     response = get_response(user_input)
17     return response.data
18
19 # Streamed response emulator
20 def respo
```

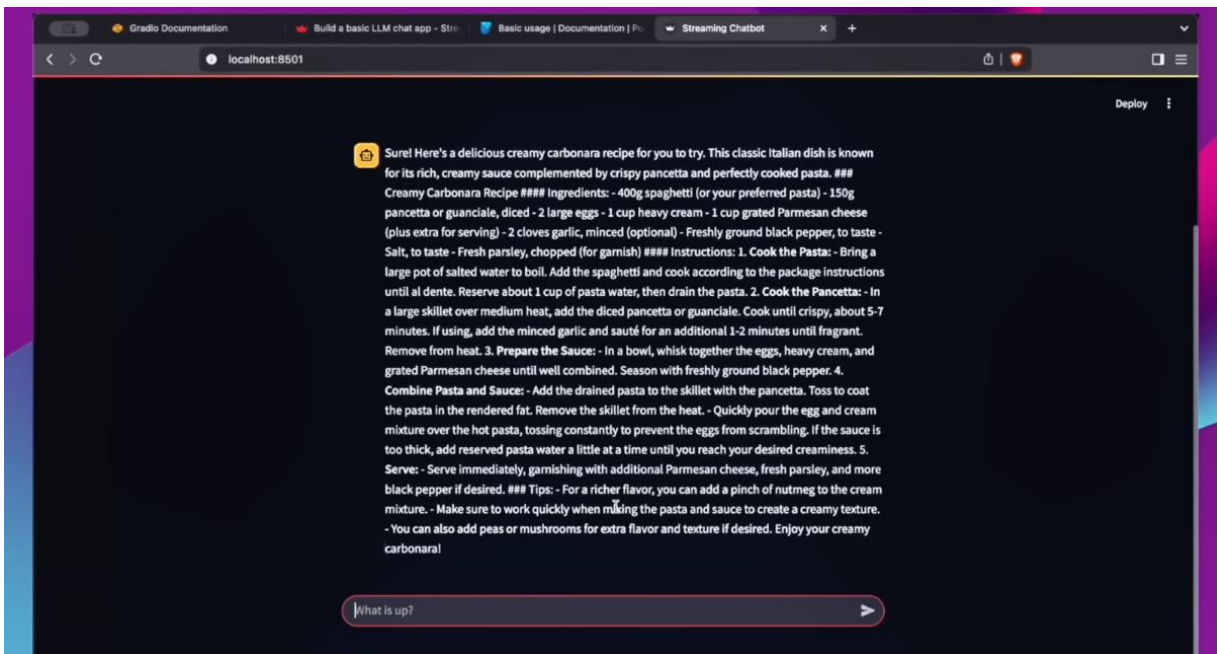
```
15 def generate_message(user_input):
16     response = get_response(user_input)
17     return response.data
18
19 # Streamed response emulator
20 def response_generator():
21     response = generate_message(prompt if prompt is not None else "Hello.")
22     for word in response.split():
23         yield word + " "
24         time.sleep(0.05)
25
26 # Initialize chat history
27 if "messages" not in st.session_state:
28     st.session_state.messages = []
29
30 # Display chat messages from history on app rerun
31 for message in st.session_state.messages:
32     with st.chat_message(message["role"]):
33         st.markdown(message["content"])
34
35 # Accept user input
36 if prompt := st.chat_input("What is up?"):
37     |
```

```
28 st.session_state.messages = []
29
30 # Display chat messages from history on app rerun
31 for message in st.session_state.messages:
32     with st.chat_message(message["role"]):
33         st.markdown(message["content"])
34
35 # Accept user input
36 if prompt := st.chat_input("What is up?"):
37     # Display user message in chat message container
38     with st.chat_message("user"):
39         st.markdown(prompt)
40
41     # Add user message to chat history
42     st.session_state.messages.append({"role": "user", "content": prompt})
43
44 # Display assistant respon
```

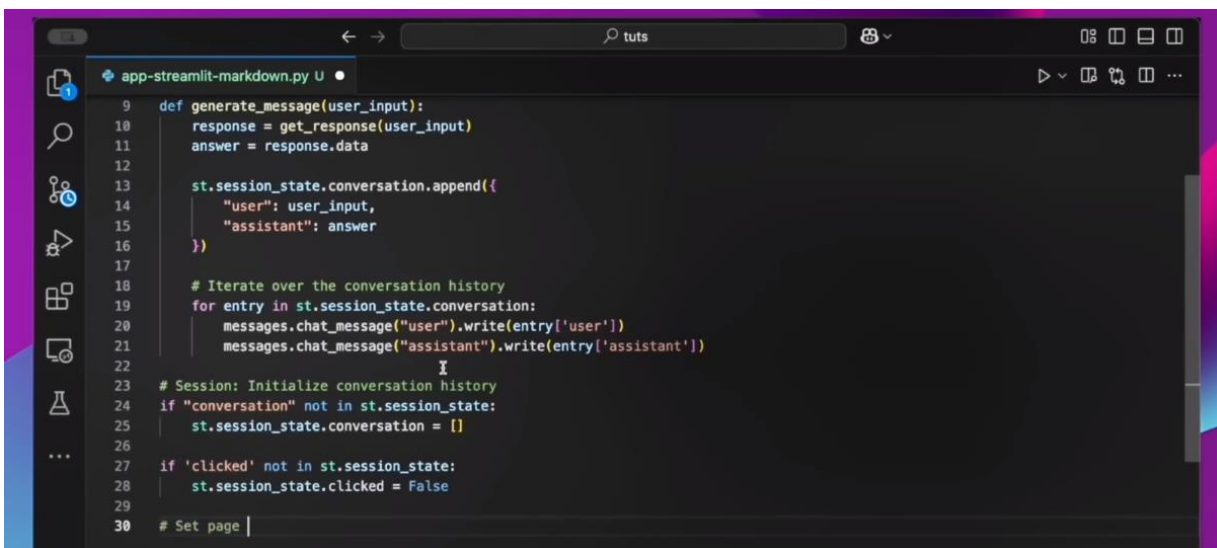
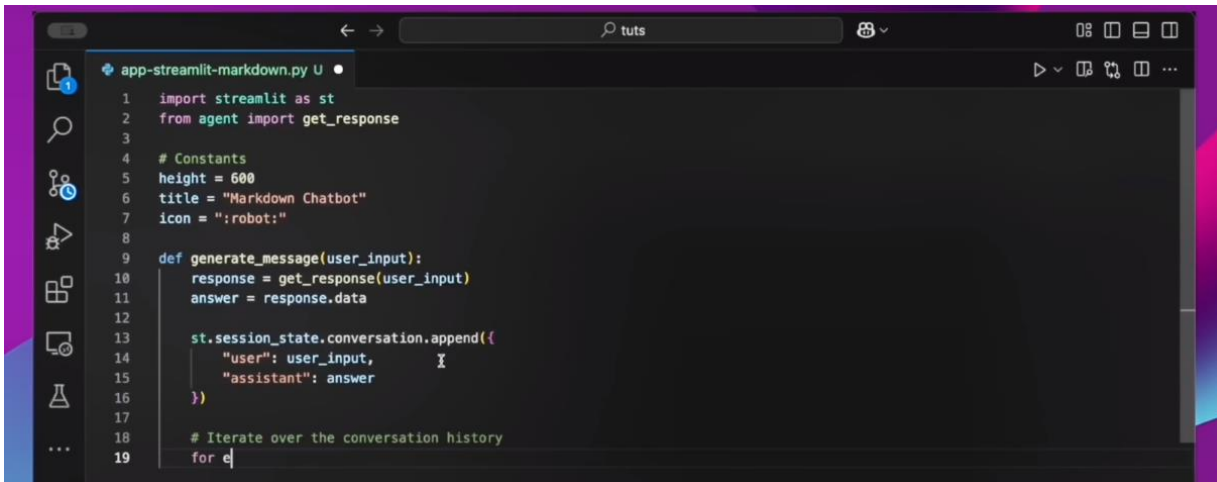
```
39 | st.markdown(prompt)
40 |
41 | # Add user message to chat history
42 | st.session_state.messages.append({"role": "user", "content": prompt})
43 |
44 | # Display assistant response in chat message container
45 | with st.chat_message("assistant"):
46 |     response = st.write_stream(response_generator())
47 |
48 | # Add assistant response to chat history
49 | st.session_state.messages
```

```
41 | # Add user message to chat history
42 | st.session_state.messages.append({"role": "user", "content": prompt})
43 |
PROBLEMS 12 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS python3.12 - 6-agent-ui + - - - ^ x
(mynv) 18:11:30 ~/dev/tuts/full-stack-ai-masterclass/6-agent-ui $ poetry run streamlit run app-streamlit-stream.py
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.1.89:8501
```





This version of the streaming **Streamlit** framework cannot show markdown together with text



```
app-streamlit-markdown.py U •
23 # Session: Initialize conversation history
24 if "conversation" not in st.session_state:
25     st.session_state.conversation = []
26
27 if 'clicked' not in st.session_state:
28     st.session_state.clicked = False
29
30 # Set page title and icon
31 st.set_page_config(page_title=title, page_icon=icon)
32 st.header(title)
33
34 messages = st.container(border=True, height=height)
35 if prompt := st.chat_input("Enter your question...", key="prompt"):
36     generate_message(prompt)
```

```
app-streamlit-markdown.py X
23 # Session: Initialize conversation history
24 if "conversation" not in st.session_state:
25     st.session_state.conversation = []
26
27 if 'clicked' not in st.session_state:
28     st.session_state.clicked = False
29
30 # Set page title and icon
31 st.set_page_config(page_title=title, page_icon=icon)
32 st.header(title)
33
34 messages = st.container(border=True, height=height)
35 if prompt := st.chat_input("Enter your question...", key="prompt"):
36     generate_message(prompt)
```

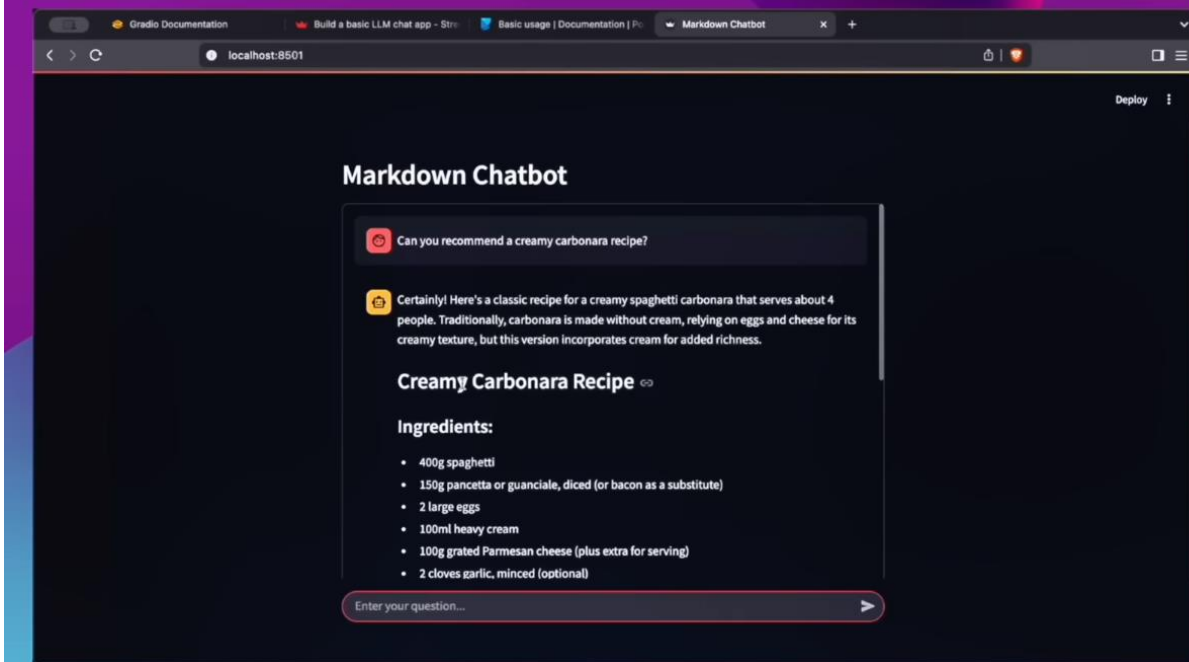
PROBLEMS (12) OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS python3.12 - 6-agent-ui + - - - ^ X

(myenv) 18:13:01 ~/dev/tuts/full-stack-ai-masterclass/6-agent-ui \$ poetry run streamlit run app-streamlit-markdown.py

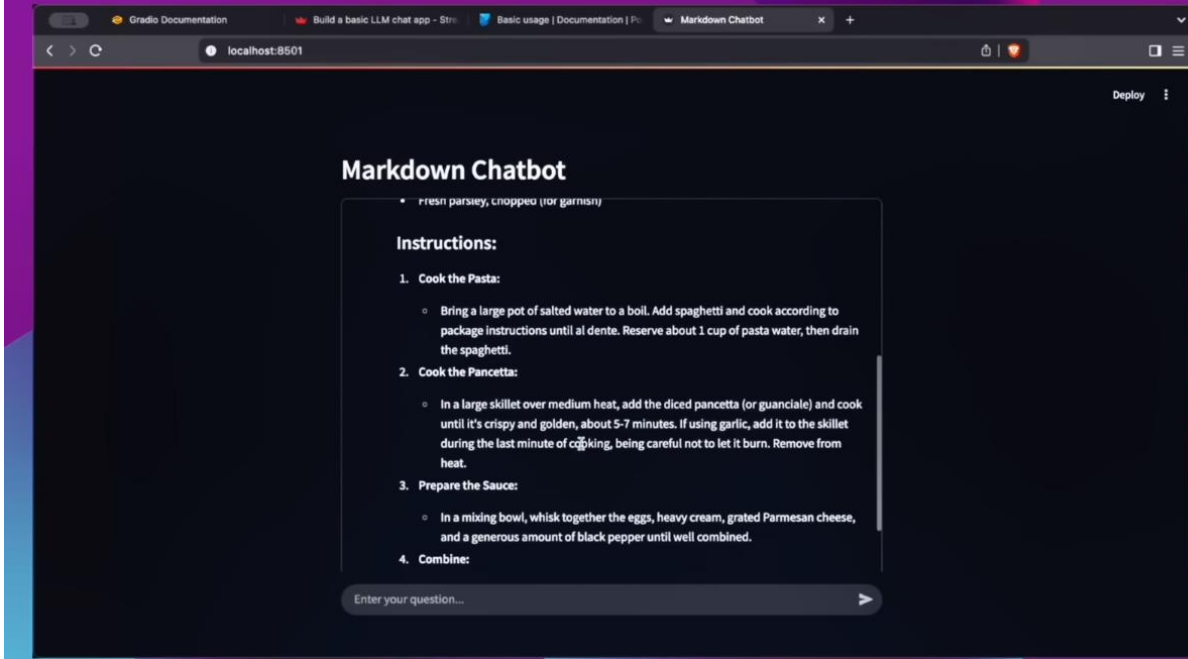
You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
Network URL: <http://192.168.1.89:8501>

Streamlit UI (Markdown)

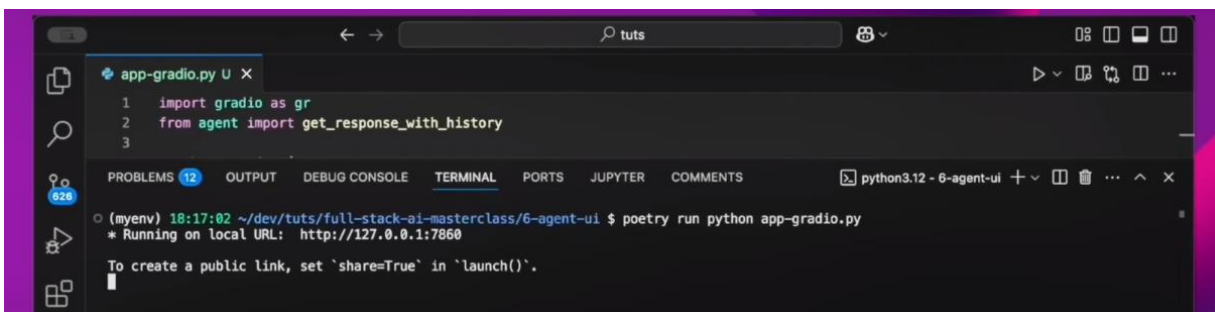
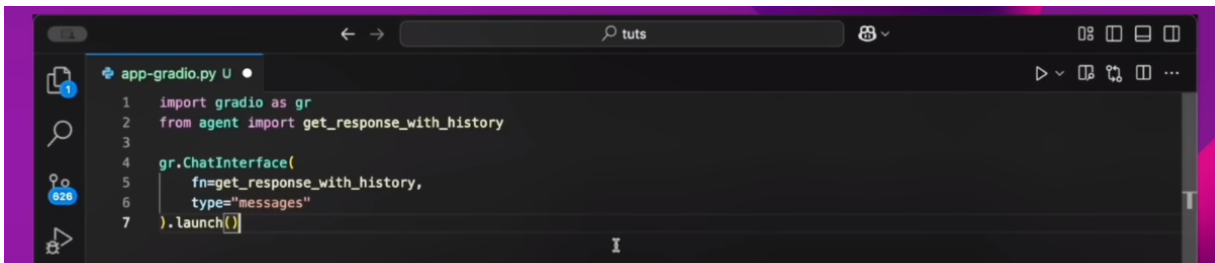


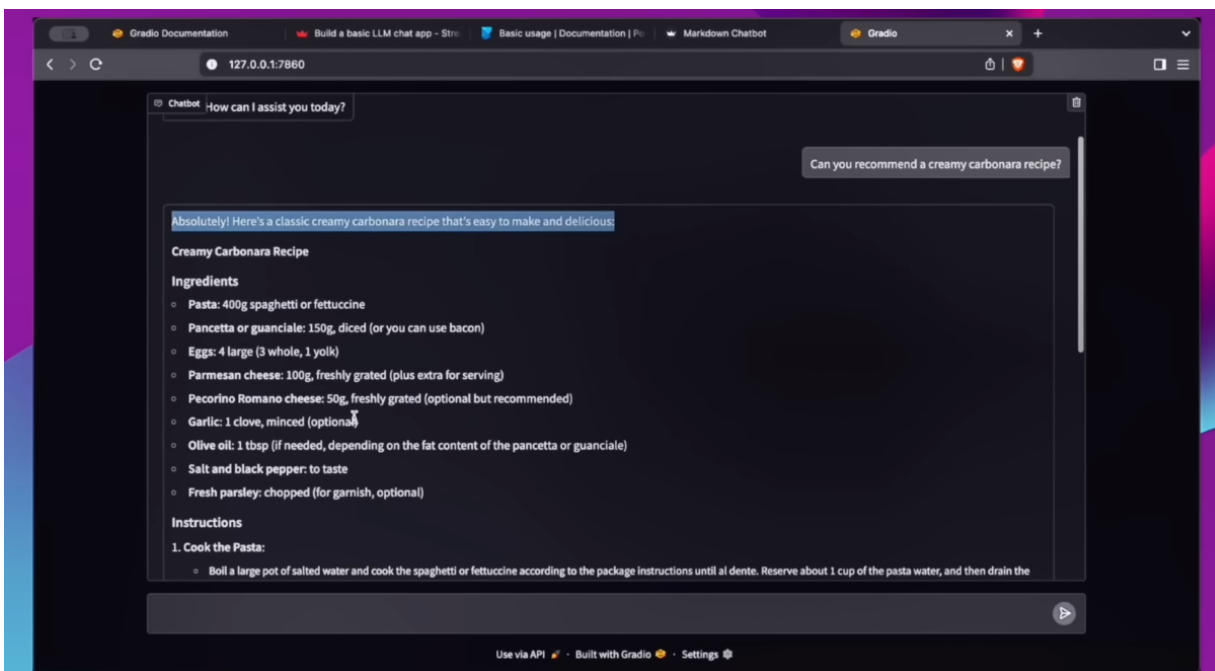
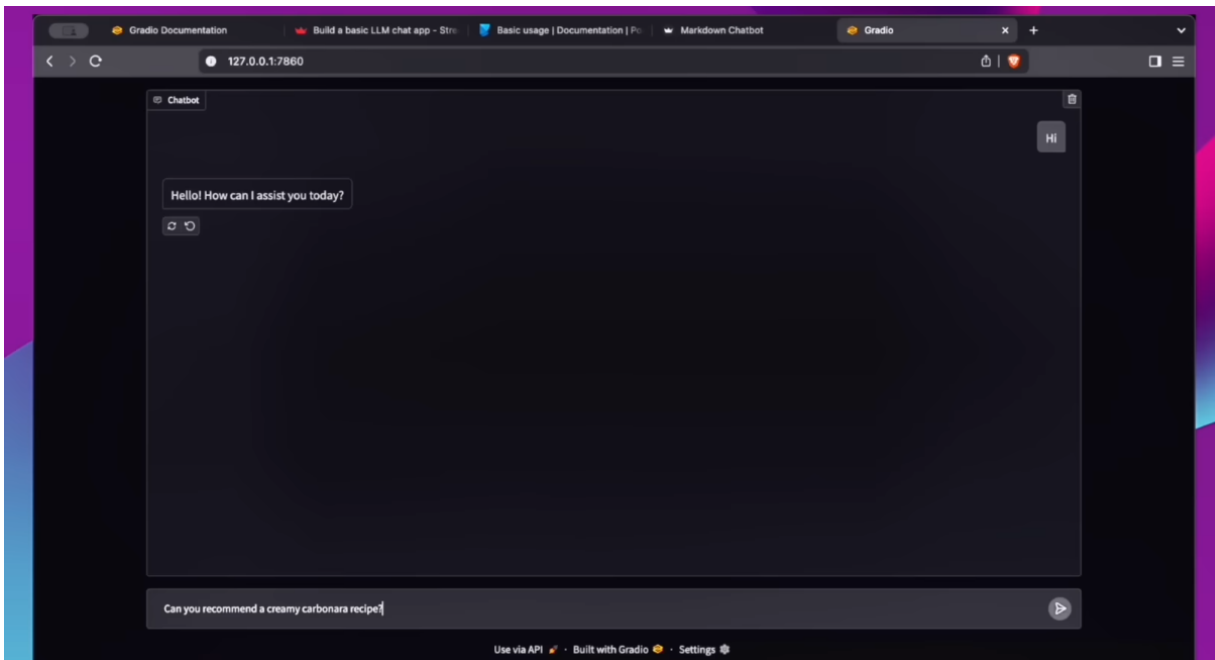
Streamlit UI (Markdown)

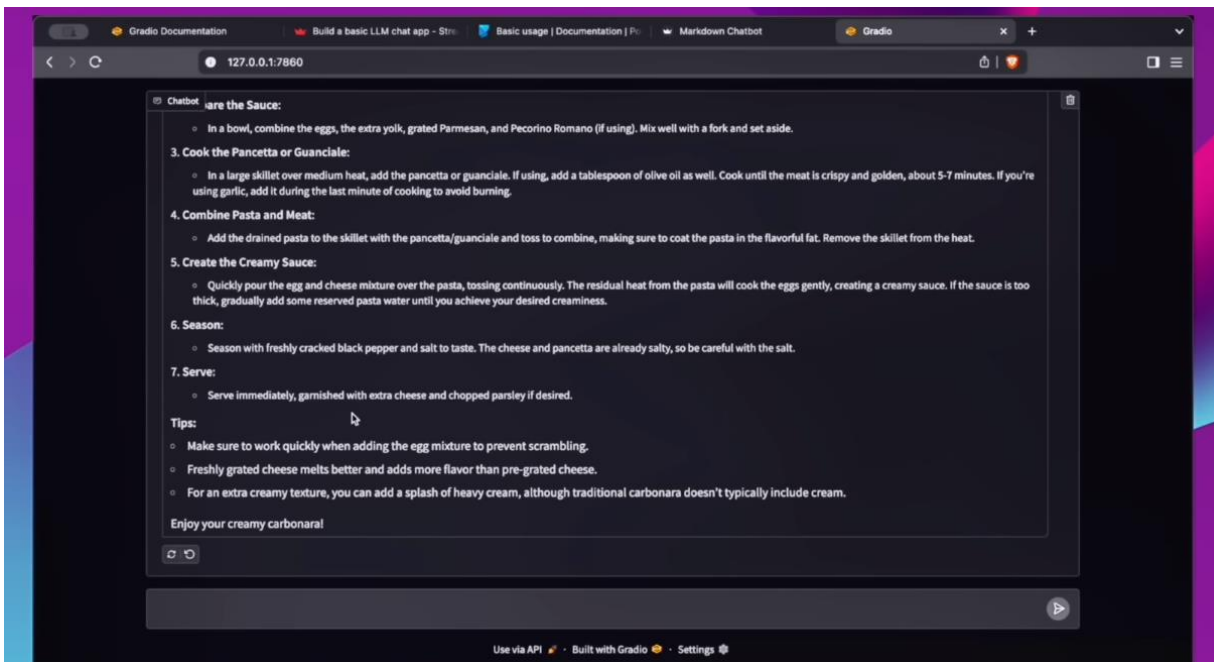


This is the markdown, non-streaming Streamlit UI

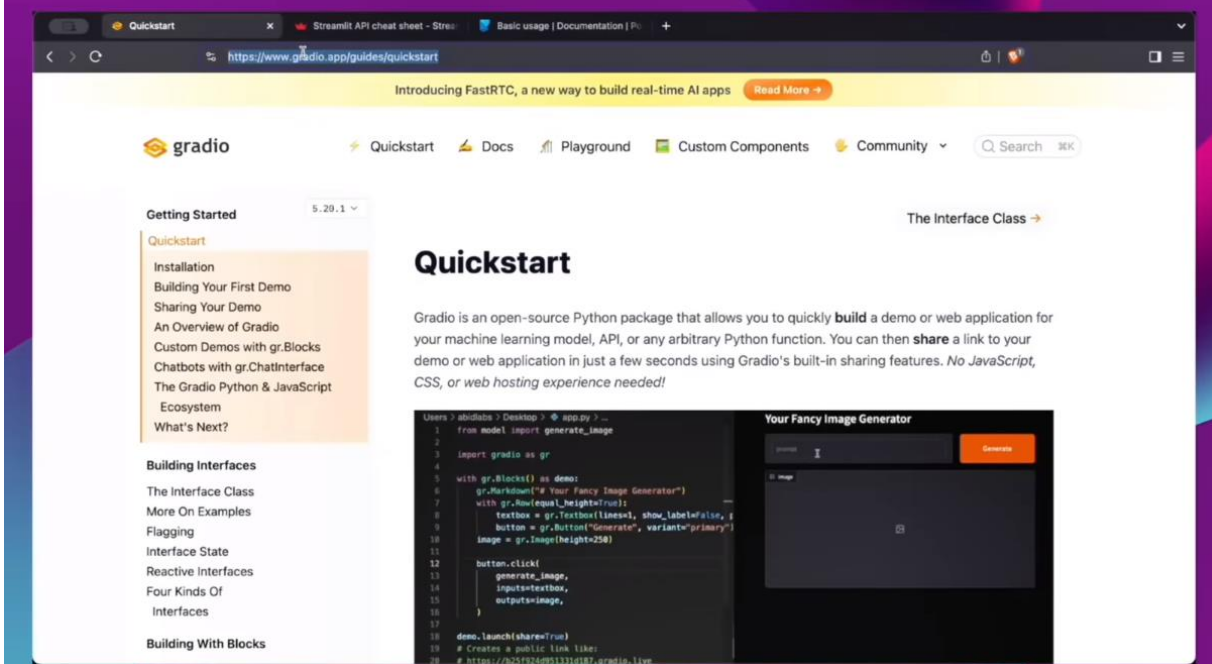
Bonus: Gradio UI (Markdown)







Gradio Documentation



Gradio Documentation

The screenshot shows the Gradio documentation website at `gradio.app/guides/quickstart`. The left sidebar contains a table of contents with sections like 'Getting Started', 'Building Interfaces', and 'Building With Blocks'. The main content area is titled 'Installation' and includes a 'Prerequisite' section stating that Gradio requires Python 3.10 or higher. It recommends installing Gradio using `pip` and provides a code block for the installation command: `pip install --upgrade gradio`. A tip box suggests installing Gradio in a virtual environment. Below the installation section, the 'Building Your First Demo' section begins, explaining that Gradio can be run in various code editors and introducing the first Gradio app.

Installation

Prerequisite: Gradio requires [Python 3.10 or higher](#).

We recommend installing Gradio using `pip`, which is included by default in Python. Run this in your terminal or command prompt:

```
pip install --upgrade gradio
```

Tip: It is best to install Gradio in a virtual environment. Detailed installation instructions for all common operating systems [are provided here](#).

Building Your First Demo

You can run Gradio in your favorite code editor, Jupyter notebook, Google Colab, or anywhere else you write Python. Let's write your first Gradio app:

```
import gradio as gr

def greet(name, intensity):
    return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
    fn=greet,
```

Gradio Documentation

This screenshot shows the same Gradio documentation website, but now displaying the 'Building Your First Demo' section. The sidebar remains the same. The main content area continues from the previous section, showing the completion of the first Gradio app code. The code defines a `greet` function and creates a `demo` interface with a text input, a slider for intensity, and a text output. The code concludes with `demo.launch()`. A tip box explains the convention of shortening `gradio` to `gr` for better readability. Below the code, instructions are given on how to run the code, either by executing `python app.py` in the terminal or by opening a browser at `http://localhost:7860`.

Building Your First Demo

You can run Gradio in your favorite code editor, Jupyter notebook, Google Colab, or anywhere else you write Python. Let's write your first Gradio app:

```
import gradio as gr

def greet(name, intensity):
    return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "slider"],
    outputs=["text"],
)

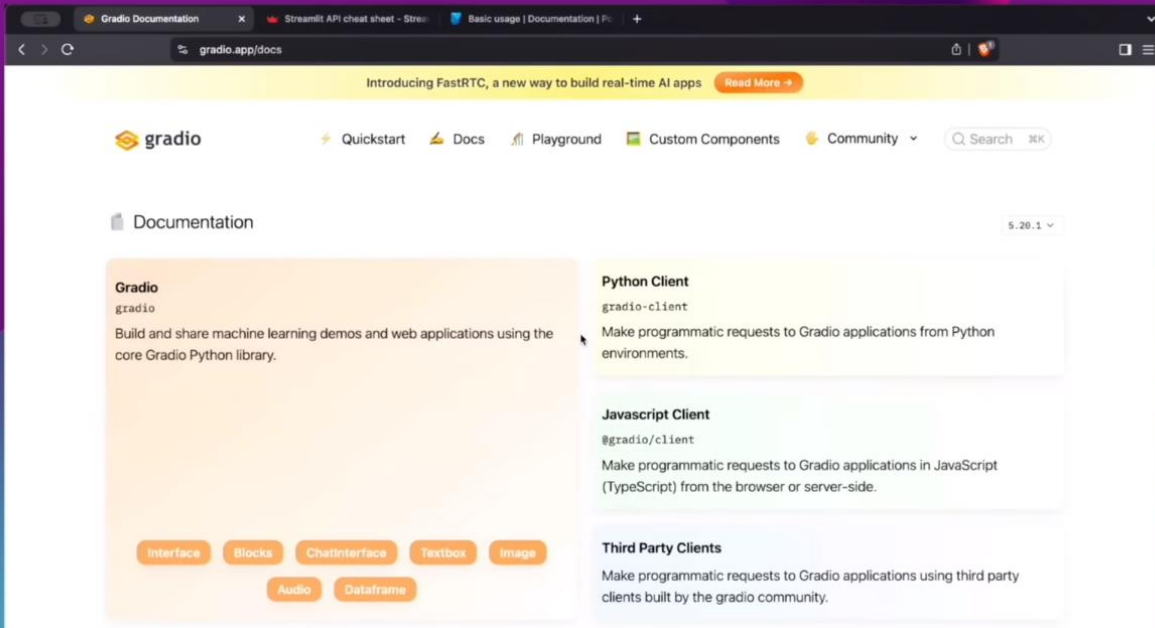
demo.launch()
```

Tip: We shorten the imported name from `gradio` to `gr`. This is a widely adopted convention for better readability of code.

Now, run your code. If you've written the Python code in a file named `app.py`, then you would run `python app.py` from the terminal.

The demo below will open in a browser on `http://localhost:7860` if running from a file. If you are running

Gradio Documentation



Summary

Building Agent UI with Streamlit and PydanticAI

AI agents are powerful, but without a **good UI**, they stay stuck in the backend. That's where **Streamlit** comes in—turning them into interactive, user-friendly applications with real-time inputs and feedback.

Today, we've explored how to build UIs for PydanticAI agents, covering structure, best practices, and a hands-on example.

Using PydanticAI and Streamlit, along with other frameworks such as Gradio, can elevate your agents and make your apps stand out.

